TRƯỜNG ĐẠI HỌC KIẾN TRÚC ĐÀ NẴNG (DAU)

Chương 2: Lập trình đa kiểu

Bài giảng · Chương 2

Học thật · làm thật · code thật

TS. Đỗ Phúc Hảo – Giảng viên *Lecturer*

Khoa Công nghệ Thông tin, Trường Đại học Kiến trúc Đà Nẵng

haodp.ai@gmail.com · 0365 674 592 · Đà Nẵng, 2026

Nội dung buổi học *Lecture outline*

DAU

1. Lập trình chức năng *functional*
- Giao tiếp hàm, 4 giao tiếp chuẩn
- Tham chiếu phương thức `::`

2. Lập trình tổng quát *generics*
- Lớp, phương thức tổng quát
- Kiểu giới hạn, wildcard, type erasure

3. Biểu thức Lambda *lambda*
- Cú pháp, bắt biến
- So với lớp ẩn danh

4. Streams API *streams*
- Trung gian, kết thúc, tính lười
- `Collectors`

Hành vi cũng là một giá trị: gói gọn rồi truyền đi như số hay chuỗi. Slide tiếng Việt là chính, code Java comment tiếng Anh.



1

Lập trình chức năng

functional programming

Hành vi là một giá trị *Behaviour as a value*

DAU

- Hướng đối tượng truyền đi *đối tượng*: dữ liệu kèm hành vi.
- Lập trình chức năng: chính *hành vi* cũng là giá trị, đặt tên, lưu biến, truyền tham số, trả về.
- Nhiều bài toán = khung xử lý cố định + một mẩu logic thay đổi.

GHI NHỚ · KEY POINT

Tách khung xử lý khỏi logic cụ thể giúp mã ngắn hơn và dễ tái dùng hơn.

Từ Java 8: gói một mẩu hành vi dưới dạng thể hiện của một *giao tiếp hàm*.

Giao tiếp hàm *Functional interface*

DAU

ĐỊNH NGHĨA · DEFINITION

Giao tiếp hàm *functional interface* là một `interface` có *đúng một* phương thức trừu tượng *single abstract method*. Nhờ vậy có thể thay thể hiện bằng Lambda hoặc tham chiếu phương thức.

- Vẫn được thêm `default` và `static`: không tính vào số SAM.
- Chú thích `@FunctionalInterface` để biên dịch kiểm tra giúp.

`Runnable`, `Comparator`, `Comparable` đã có sẵn từ trước Java 8 và tự động là giao tiếp hàm.

Tự viết một giao tiếp hàm *Custom functional interface*

DAU

```java
@FunctionalInterface
interface Validator {
   boolean isValid(String input); // the single abstract method

   // default methods do not count toward the SAM rule
   default Validator and(Validator other) {
      return s -> this.isValid(s) && other.isValid(s); // compose two rules
   }
}
```

GHI NHỚ · KEY POINT

Nhận ra giao tiếp hàm: chỉ *một* phương thức cần cài. Số `default` và `static` không giới hạn.

Bốn giao tiếp hàm chuẩn *java.util.function*

DAU

Giao tiếp	Phương thức	Kiểu	Ý nghĩa
`Function<T,R>`	`R apply(T)`	`T`→`R`	Biến đổi `T` thành `R`.
`Predicate<T>`	`boolean test(T)`	`T`→`boolean`	Kiểm tra một điều kiện.
`Consumer<T>`	`void accept(T)`	`T`→(không)	Nhận một `T`, không trả gì.
`Supplier<T>`	`T get()`	(không)→`T`	Cung cấp một giá trị `T`.

`Function` có cả vào và ra; `Predicate` là dạng riêng trả `boolean`.

`Consumer` chỉ *nhận*; `Supplier` chỉ *cho*.

Dùng bốn giao tiếp chuẩn *Standard interfaces in action*

DAU

```java
import java.util.function.*;

Function<String, Integer> length = s -> s.length(); // String -> Integer
Predicate<Integer> isEven = n -> n % 2 == 0; // Integer -> boolean
Consumer<String> printer = s -> System.out.println(s); // String -> void
Supplier<String> greeter = () -> "Hello, DAU"; // () -> String

int n = length.apply("functional"); // n = 10
boolean even = isEven.test(n); // even = true
String msg = greeter.get(); // msg = "Hello, DAU"
```

Ghép nối hành vi *Composing behaviour*


```
Function<String, String> trim = String::strip;
Function<String, String> upper = String::toUpperCase;
Function<String, String> clean = trim.andThen(upper); // trim first, then upper
System.out.println(clean.apply(" dau ")); // prints: DAU

Predicate<Integer> positive = n -> n > 0;
Predicate<Integer> small = n -> n < 100;
Predicate<Integer> inRange = positive.and(small); // 0 < n < 100
```

`Function`: `andThen`, `compose`. `Predicate`: `and`, `or`, `negate`. Lắp mẩu nhỏ thành hành vi lớn, không cần lớp mới.

Tham chiếu phương thức *Method reference*

- Khi Lambda chỉ gọi lại đúng một phương thức đã có, viết hẳn Lambda là thừa.
- Trỏ thẳng tới phương thức qua cú pháp hai dấu hai chấm `::`.
- `s -> s.length()` viết gọn thành `String::length`.

Dạng	Cú pháp	Lambda tương đương
Phương thức tĩnh	`Integer::parseInt`	`s -> Integer.parseInt(s)`
Của một đối tượng cụ thể	`System.out::println`	`x -> ...println(x)`
Của một đối tượng bất kỳ thuộc kiểu	`String::toUpperCase`	`s -> s.toUpperCase()`
Hàm dựng	`ArrayList::new`	`() -> new ArrayList<>()`

Bốn dạng tham chiếu *Four reference forms*

DAU

```java
Function<String, Integer> parse = Integer::parseInt; // static method
Consumer<String> print = System.out::println; // bound instance method
Function<String, String> upper = String::toUpperCase; // unbound instance method
Supplier<List<String>> maker = ArrayList::new; // constructor reference

System.out.println(parse.apply("17") + 1); // prints: 18
print.accept(upper.apply("dau")); // prints: DAU
```

LỖI HAY GẶP · PITFALL

`String::toUpperCase` là một *giá trị*, gọi về sau qua `apply`. Đừng nhầm với `"dau".toUpperCase()` vốn chạy ngay.

2

Lập trình tổng quát

generic programming

Vì sao cần kiểu tổng quát *Why generics*

DAU

Một hộp chứa giá trị kiểu `Object`:

- Lấy ra phải ép kiểu thủ công; ép sai ném `ClassCastException` *lúc chạy*.
- Không gì ngăn bỏ chuỗi vào hộp lẽ ra chứa số.

ĐỊNH NGHĨA · DEFINITION

Lập trình tổng quát *generic* viết lớp, giao tiếp, phương thức với một *tham số kiểu* chưa xác định (`T`, `E`, `K`, `V`), kiểm tra đúng kiểu lúc biên dịch.

Quy ước tên: `T` type, `E` element, `K`/`V` key/value, `R` result.

Lớp tổng quát *Generic class*

DAU

```java
public class Box<T> { // T is a type parameter
   private T value;
   public Box(T value) { this.value = value; }
   public T get() { return value; } // returns the exact type T
   public void set(T v) { this.value = v; }
}

Box<String> sb = new Box<>("DAU"); // T is fixed to String here
String s = sb.get(); // no cast needed, type-safe
// sb.set(99); // compile error: 99 is not a String
```

GHI NHỚ · KEY POINT

So với `Object`: loại bỏ ép kiểu, đẩy mọi kiểm tra kiểu về lúc biên dịch.

Phương thức tổng quát *Generic method*


```java
public class Utils {
   // <T> declares the method's own type parameter before the return type
   public static <T> T firstOf(List<T> items) {
      return items.isEmpty() ? null : items.get(0);
   }
}

List<String> names = List.of("An", "Binh");
String first = Utils.firstOf(names); // compiler infers T = String
```

Tham số kiểu riêng đặt ngay *trước* kiểu trả về. Trình biên dịch tự suy `T` từ đối số: **suy luận kiểu** *type inference*.

Kiểu giới hạn với extends *Bounded type*

DAU

```java
// T must be a subtype of Comparable<T>, so compareTo is available
public static <T extends Comparable<T>> T max(List<T> items) {
   T best = items.get(0);
   for (T x : items) {
      if (x.compareTo(best) > 0) best = x; // legal thanks to the bound
   }
   return best;
}
Integer m = max(List.of(3, 9, 1, 7)); // m = 9
```

LỖI HAY GẶP · PITFALL

Từ khoá luôn là `extends`, kể cả khi giới hạn là một giao tiếp. Ở đây nghĩa rộng là “là kiểu con của”.

Ký tự đại diện wildcard *Wildcard*

DAU

LỖI HAY GẶP · PITFALL

`List<Integer>` *không phải* kiểu con của `List<Number>`. Generics *bất biến* *invariant* theo tham số kiểu.

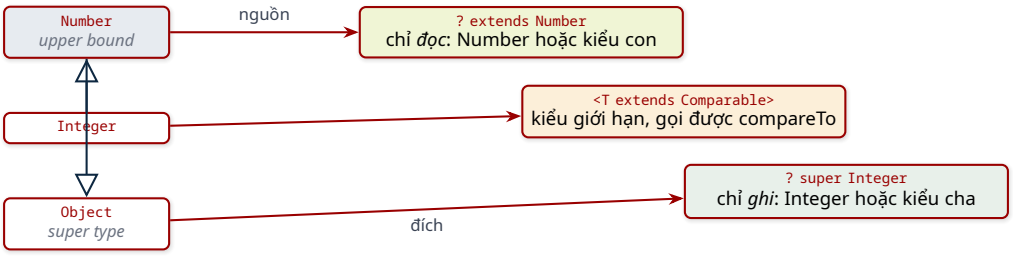
- `<? extends T>` giới hạn trên: *đọc* ra.
- `<? super T>` giới hạn dưới: *ghi* vào.

GHI NHỚ · KEY POINT

PECS: producer extends, consumer super. Nguồn để đọc dùng `extends`; đích để ghi dùng `super`.

Không đọc cũng không ghi theo kiểu cụ thể: dùng `<?>`.

Sơ đồ kiểu giới hạn và wildcard *Bounds and wildcards*



Type erasure *Xoá kiểu*

- Thông tin tham số kiểu *không tồn tại lúc chạy*.
- `T` không giới hạn → `Object`; `T extends Number` → `Number`.
- Tương thích ngược với thư viện cũ chưa có generics.

LỖI HAY GẶP · PITFALL

Không biên dịch hoặc vô nghĩa lúc chạy: `instanceof List<String>`, `new T[n]`, `T.class`.

Cần kiểu lúc chạy: truyền thêm `Class<T>`.



3

Biểu thức Lambda

lambda expression

Lambda là gì *What is a lambda*

DAU

- Cách cũ truyền hành vi: tạo *lớp ẩn danh* cài đặt giao tiếp, dài dòng.
- Lambda cắt bỏ khung thừa, chỉ giữ tham số và thân hàm.

ĐỊNH NGHĨA · DEFINITION

Biểu thức Lambda là một hàm vô danh viết gọn `(tham số) -> thân hàm`. Nó là một thể hiện của một giao tiếp hàm.

Ba phần: danh sách tham số trong ngoặc, mũi tên `->`, thân hàm (biểu thức đơn hoặc khối lệnh).

Cú pháp Lambda *Lambda syntax*

DAU

```
Function<Integer, Integer> square = x -> x * x; // one param, expression body
Consumer<String> printer = s -> System.out.println(s); // parentheses optional
BinaryOperator<Integer> add = (a, b) -> a + b; // multiple params need ()
Supplier<Double> rng = () -> Math.random(); // no param: empty ()

Comparator<String> byLength = (a, b) -> { // block body
   int diff = a.length() - b.length(); // compute the gap
   return Integer.compare(diff, 0); // need return in a block
};
```

Một tham số không ghi kiểu: bỏ luôn ngoặc đơn. Thân là biểu thức đơn: không viết `return`, giá trị biểu thức là kết quả.

Bắt biến *Variable capture*

DAU

```java
int base = 100; // effectively final
Function<Integer, Integer> addBase = x -> x + base; // captures base
System.out.println(addBase.apply(5)); // prints 105

// base = 200; // COMPILE ERROR: base would no longer be effectively final
```

GHI NHỚ · KEY POINT

Biến cục bộ bị bắt phải **hữu hiệu bất biến** *effectively final*: gán một lần, không gán lại.

Lambda có thể chạy về sau, ở luồng khác. Java sao chép giá trị nên buộc biến bất biến để tránh lệch.

So với lớp ẩn danh *Versus anonymous class*

DAU

```java
// (a) anonymous class: verbose, the real logic is one line
names.sort(new Comparator<String>() {
    @Override
    public int compare(String a, String b) { return a.length() - b.length(); }
});

// (b) lambda: same behaviour, the boilerplate disappears
names.sort((a, b) -> a.length() - b.length());
```

Khác: trong Lambda `this` trỏ đối tượng lớp *bao quanh*; lớp ẩn danh sinh tệp `.class` riêng, Lambda dùng `invokedynamic` nhẹ hơn.

Khi nào dùng cái nào *Lambda or anonymous*

DAU

Dùng Lambda
khi đích là giao tiếp hàm và chỉ cần một thân hàm ngắn.

Giữ lớp ẩn danh
khi cần nhiều phương thức, trường trạng thái riêng, hoặc `this` trỏ chính nó.

GHI NHỚ · KEY POINT

Cả hai cùng tạo một đối tượng cài đặt giao tiếp. Lambda chỉ lược bỏ tên lớp, tên phương thức, `@Override` và ngoặc thừa.



4

Streams API

xử lý dữ liệu theo dây chuyền

Stream là gì *What is a stream*

- Một chuỗi phần tử đi qua một loạt phép xử lý theo phong cách dây chuyền.
- Mô tả *cái cần làm*, Java lo phần thực thi: mã gần với ý định.

LỖI HAY GẶP · PITFALL

Stream không lưu trữ phần tử, dùng một lần. Sau thao tác kết thúc, gọi lại ném `IllegalStateException`. Không đổi nguồn gốc.

Tạo stream *Creating a stream*


```java
List<Integer> nums = List.of(3, 1, 4, 1, 5, 9, 2, 6);

Stream<Integer> s1 = nums.stream(); // from a collection
Stream<String> s2 = Stream.of("a", "b", "c"); // from explicit values
IntStream s3 = IntStream.rangeClosed(1, 10); // a range of int
Stream<Integer> s4 = Arrays.stream(new Integer[]{7, 8, 9}); // from an array
```

`IntStream`, `LongStream`, `DoubleStream` chuyên cho kiểu nguyên thuỷ: tránh đóng hộp *boxing*, có sẵn `sum()`, `average()`.

Thao tác trung gian *Intermediate operations*

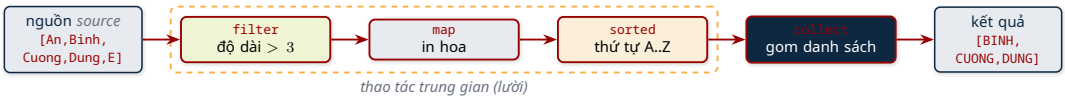
DAU

```java
List<String> names = List.of("An", "Binh", "Cuong", "Dung", "E");

List<String> result = names.stream()
    .filter(n -> n.length() > 3) // keep names longer than 3 chars
    .map(String::toUpperCase) // transform each to upper case
    .sorted() // sort by natural order
    .collect(Collectors.toList()); // gather into a new list
// result = [BINH, CUONG, DUNG]
```

`filter(Predicate)` giữ phần tử thoả; `map(Function)` biến đổi từng phần tử; `sorted()` sắp xếp. Mỗi thao tác trả về một `Stream` mới nên nối được.

Dây chuyền Stream *Stream pipeline*



Dữ liệu chảy từ nguồn, qua các trạm trung gian, tới thao tác kết thúc gom kết quả.

Thao tác kết thúc *Terminal operations*

DAU

```java
List<Integer> nums = List.of(3, 1, 4, 1, 5, 9, 2, 6);

// reduce: combine all elements into one value (the sum)
int total = nums.stream().reduce(0, (a, b) -> a + b); // 31

// collect: gather even numbers into a new list
List<Integer> evens = nums.stream()
    .filter(n -> n % 2 == 0)
    .collect(Collectors.toList()); // [4, 2, 6]

// forEach: a side effect for each element, returns nothing
nums.stream().distinct().forEach(System.out::println);
```

Tính lười *Laziness*

- Thao tác trung gian chỉ ghi nhận ý định, không chạy ngay.
- Dây chuyền khởi động khi gặp một thao tác *kết thúc*.
- Thiếu thao tác kết thúc: không phần tử nào được xử lý.

GHI NHỚ · KEY POINT

Java xử lý từng phần tử đi suốt dây chuyền rồi mới tới phần tử kế. `findFirst`, `limit` dừng sớm khi đủ.

Quan sát tính lười *Laziness in action*

DAU

```java
List<Integer> r = Stream.of(1, 2, 3, 4, 5)
    .map(n -> {
       System.out.println("mapping " + n); // shows when map runs
       return n * n;
    })
    .limit(2) // stop after two elements
    .collect(Collectors.toList());
// prints only "mapping 1" and "mapping 2", not 3, 4, 5
// r = [1, 4]
```

Phân biệt qua kiểu trả về: trung gian trả `Stream` (còn nối), kết thúc trả kết quả khác (giá trị, tập hợp, `void`).

Trung gian so với kết thúc *Intermediate vs terminal*

DAU

Thao tác	Loại	Ý nghĩa
`filter`	trung gian	Giữ phần tử thoả vị từ *predicate*.
`map`	trung gian	Biến đổi từng phần tử, có thể đổi kiểu.
`sorted`	trung gian	Sắp xếp theo thứ tự tự nhiên hoặc bộ so sánh.
`distinct`	trung gian	Loại bỏ phần tử trùng lặp.
`limit`	trung gian	Giữ lại tối đa *n* phần tử đầu.
`forEach`	kết thúc	Chạy một hành động cho mỗi phần tử, không trả về.
`collect`	kết thúc	Gom phần tử vào tập hợp qua một `Collector`.
`reduce`	kết thúc	Gộp các phần tử thành một giá trị duy nhất.
`count`	kết thúc	Đếm số phần tử của stream.

Lớp Collectors *Common collectors*

DAU

```java
record Person(String name, String city) {}
List<Person> people = List.of(
    new Person("An", "Hue"), new Person("Binh", "Da Nang"),
    new Person("Cuong", "Hue"), new Person("Dung", "Da Nang"));

// joining: concatenate strings with a separator
String line = people.stream()
    .map(Person::name).collect(Collectors.joining(", ")); // "An, Binh, Cuong, Dung"

// groupingBy with a downstream collector: count members per group
Map<String, Long> countByCity = people.stream()
    .collect(Collectors.groupingBy(Person::city, Collectors.counting()));
// {Hue=2, Da Nang=2}
```

Sức mạnh của groupingBy *groupingBy in depth*

- Phân loại phần tử theo một hàm khoá rồi gom mỗi nhóm thành `Map`.
- Tham số thứ hai (*bộ gom hạ nguồn*) xử lý tiếp từng nhóm: `counting`, `summingInt`, `averagingDouble`.

Trong các thao tác trung gian, viết hàm *không tác dụng phụ*: chỉ tính dựa trên đầu vào. Mã thuần dễ kiểm thử và an toàn cho `parallelStream()`.

5

Tóm tắt & bài tập

summary & exercises

Tóm tắt chương 2 *Chapter summary*

DAU

Chức năng & tổng quát

- Giao tiếp hàm = một SAM; 4 chuẩn: `Function`, `Predicate`, `Consumer`, `Supplier`.
- Tham chiếu phương thức `::`.
- Generic, kiểu giới hạn, wildcard (PECS), type erasure.

Lambda & Streams

- Lambda `(p) -> body`, bắt biến hữu hiệu bất biến.
- Stream: trung gian (lười) so với kết thúc.
- `collect`, `reduce`, `Collectors.groupingBy`.

GHI NHỚ · KEY POINT

Một dây chuyền Stream hợp lệ luôn kết thúc bằng đúng một thao tác kết thúc.

Bài tập về nhà *Homework*

1. Giải thích vì sao giao tiếp phải là giao tiếp hàm thì mới thay được bằng Lambda. Cho ví dụ giao tiếp *không* dùng được Lambda.
2. So sánh Lambda và lớp ẩn danh: độ dài mã, ý nghĩa `this`, số tệp `.class`, trường hợp nên dùng mỗi loại.
3. (*) Viết lớp `Box<T>` có phương thức `map` nhận `Function<T,R>` và trả về `Box<R>`. Dùng Lambda biến `Box<Integer>` chứa 5 thành `Box<String>` chứa `"5"`.
4. (*) Với `List<String>`: lọc từ dài hơn bốn ký tự, đổi chữ thường, loại trùng, sắp theo bảng chữ cái, gom bằng `Collectors.joining`.
5. (*) Với `List` các `record SinhVien(String ten, String lop, double diem)`: nhóm theo lớp bằng `groupingBy`, mỗi lớp tính điểm trung bình bằng `averagingDouble`.

CẢM ƠN

Hỏi đáp · Thảo luận

TS. Đỗ Phúc Hảo

haodp.ai@gmail.com · 0365 674 592 · Khoa CNTT, ĐH Kiến trúc Đà Nẵng